전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

JST

지능형모빌리티 설계

7주차 ROS2 프로그래밍 #5

1차시 ROS2 NAV2 (1)



* 본 영상은 무단 배포할 수 없습니다.

학습내용

1 ROS2 Nav2 패키지 사용하기

학습목표

🎯 ROS2 Nav2 패키지를 통해
자율주행 로봇을 구현할 수 있다.

ROS2 Nav2 패키지

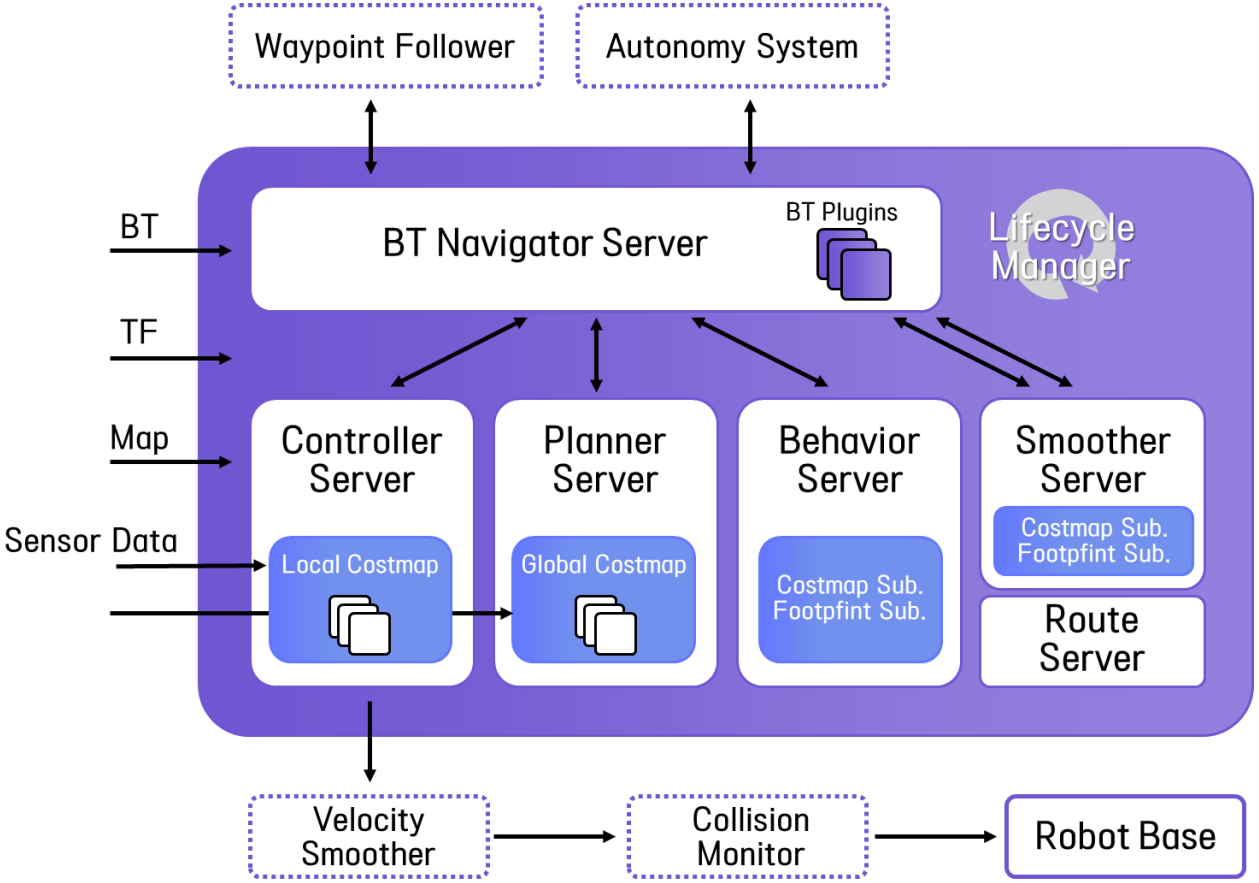
NAV2란?

NAV2 개요

- Planning, Control, Localization 등 높은 신뢰성을 가진 자율 주행 시스템 프레임워크
- 독립적인 모듈형 서버들을 조율해서 지능형 네비게이션 동작을 생성하는 Behavior Tree 사용

NAV2란?

NAV2 구조



NAV2란?

NAV2의 사용 가능한 기능

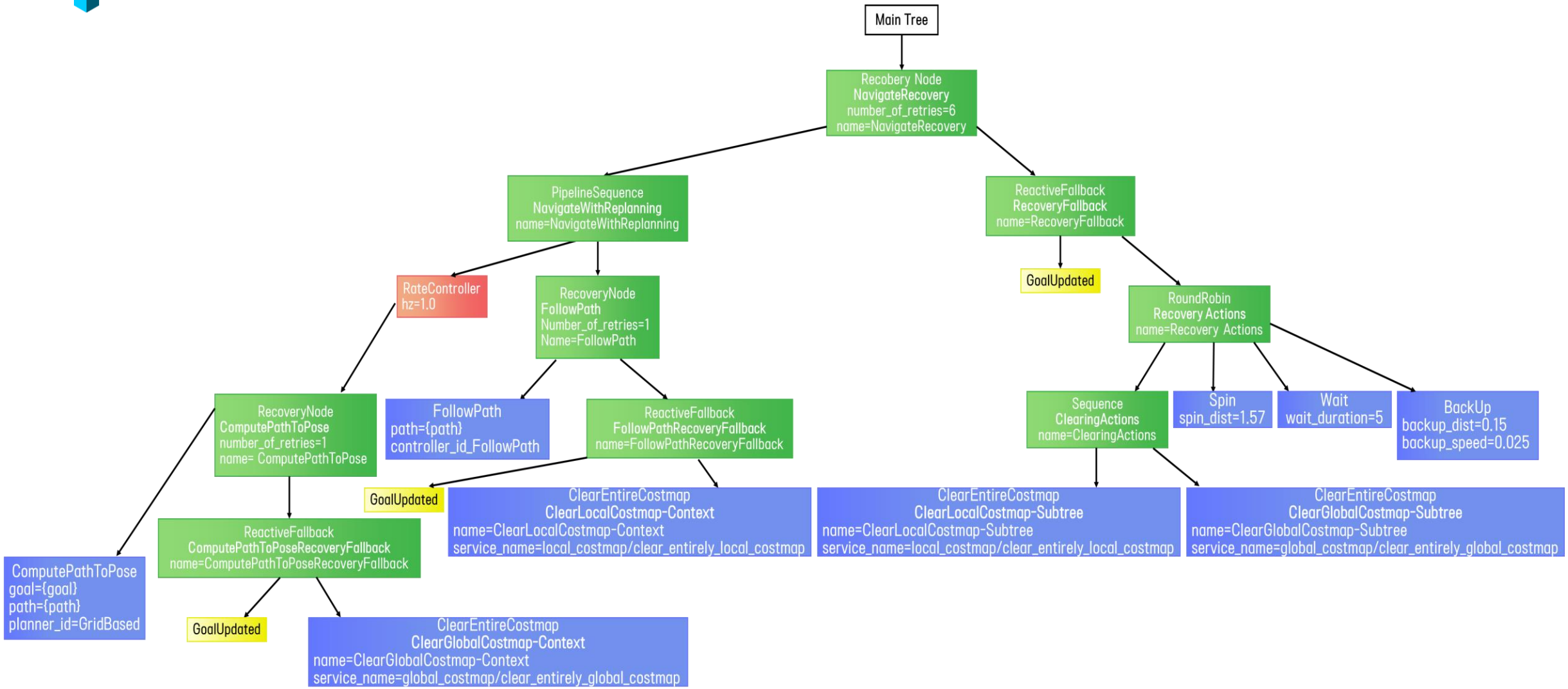
- 맵 저장, 로드
- 맵에서 위치 추정
- 충돌 회피, 경로계획
- 미리 정의된 동작 실행
- 사용자가 지정한 위치를 따라가는 웨이포인트 팔로워
- 사용자가 정의한 알고리즘에 따른 동작
- 실시간 충돌에 대비한 센서 데이터 모니터링
- 자율주행 구현에 필요한 기본적인 기능은 모두 들어있음

NAV2의 Behavior Tree

Behavior Tree란?

- 기존의 FSM은 One-way Control Transfer로써 프로그램이 다른 코드로 점프하면서 실행하는 방법
- 이는 원시적이며, 프로그램의 이해를 망치는 요인 중 하나가 됨

NAV2의 Behavior Tree



NAV2란?

NAV2의 다양한 Plugins

상황에 맞게 추가 가능한 다양한 Plugin이 존재
(https://docs.nav2.org/plugins/index.html_

- Behavior-Tree Navigators
- Costmap Layers
- Costmap Filters
- Controllers
- Planners
- Smoothers
- Waypoint Task Executors
- etc.

NAV2란?

NAV2 파라미터 튜닝

- 자율주행을 위한 환경에 따라 사용자가 파라미터를 결정해줘야 함[어려움!] (https://docs.nav2.org/configuration/index.html)
- 모바일 로봇 바퀴의 지름, 몸체 크기, 속도 등 많은 부분이 튜닝 파라미터에 관여
- 수많은 파라미터가 하나의 시스템에 들어가며 이는 숙련자일수록 빠르게 안정화 가능
  - 튜닝 가이드를 참고해서 본인만의 튜닝 기술을 찾아보자
- AMCL의 파라미터 예시
  - 오도메트리 회전 노이즈
  - 전방향 이동체 유무
  - 장애물과 최소 거리
  - 라이다 센서 유효 거리
  - 파티클 개수
  - etc.

NAV2 실습

패키지 설치

```
$ cd ~/ros2_ws/src
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git
$ cd ~/ros2_ws/src/turtlebot3
$ git clone -b humble https://github.com/ROBOTIS-
GIT/turtlebot3_msgs.git
$ git clone -b humble https://github.com/ROBOTIS-
GIT/turtlebot3_simulations.git
$ git clone -b humble https://github.com/ROBOTIS-GIT/DynamixelSDK.git
$ sudo apt install ros-humble-cartographer* -y
$ sudo apt install ros-humble-nav* -y
$ cd ~/ros2_ws && colcon build
$ source install/setup.bash
$ gedit ~/.bashrc
```

```
# ~/.bashrc 맨 아래에 붙여넣기
export TURTLEBOT3_MODEL=burger
```

NAV2 실습

지도 작성 실습

- 자율주행을 위해서 사전에 작성된 지도가 필요함
- Gazebo를 이용해 미리 모델링된 공간을 생성하고 지도를 작성
- Cartographer라는 패키지를 이용

NAV2 실습

지도 작성 실습

터미널 1

```
$ source ~/.bashrc
$ . /usr/share/gazebo/setup.sh
$ ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py
```

터미널 2

```
$ source ~/ros2_ws/install/local_setup.bash
$ ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

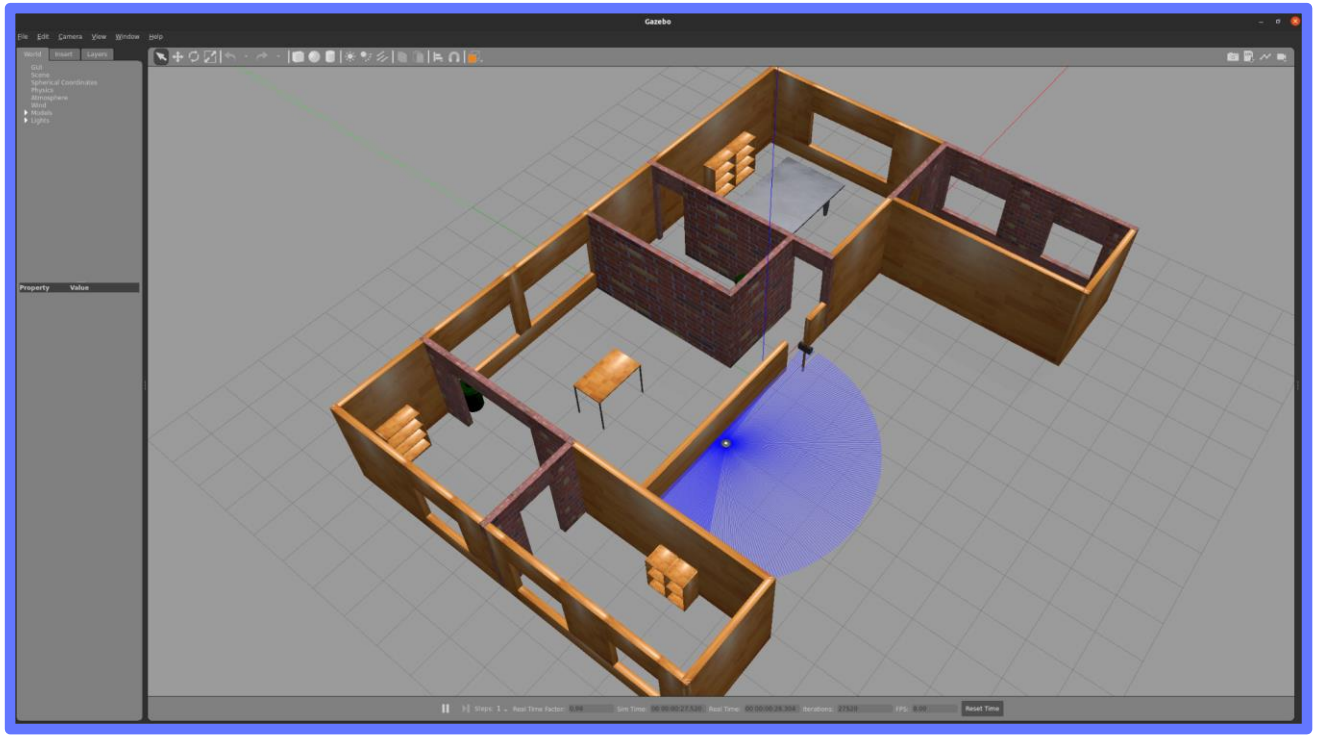
터미널 3

```
$ source ~/ros2_ws/install/local_setup.bash
$ ros2 run turtlebot3_teleop teleop_keyboard
```

NAV2 실습

지도 작성 실습

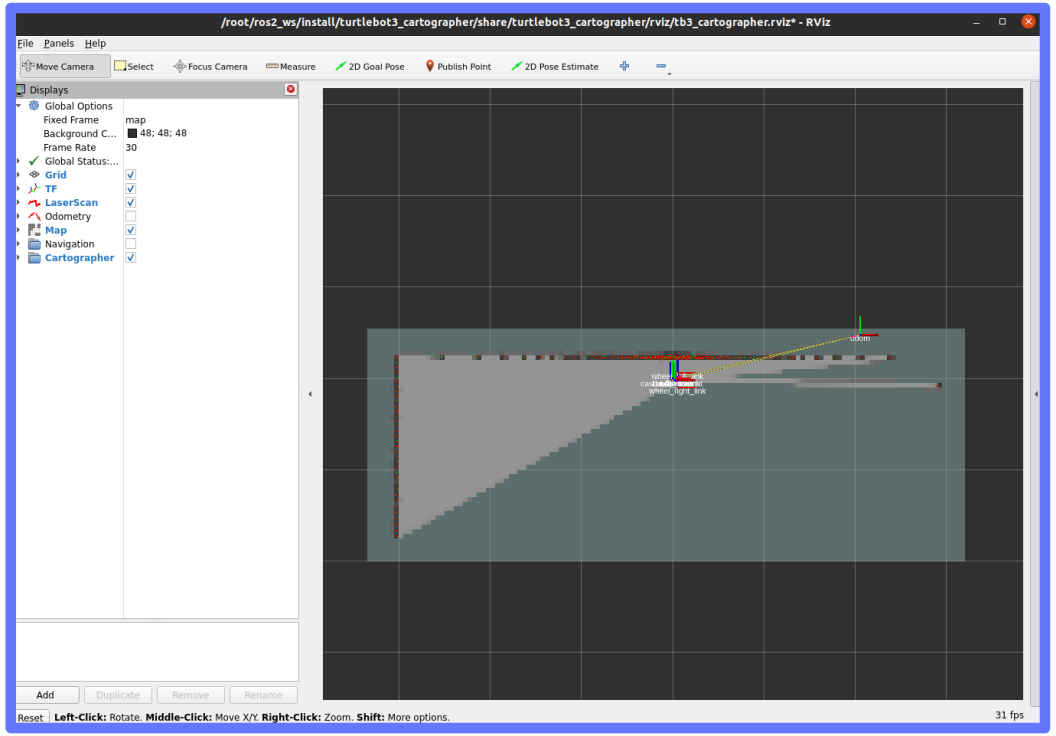
- 파란색 실선은 로봇의 2D 라이다 센서 빔



NAV2 실습

지도 작성 실습

Cartographer rviz 시각화



NAV2 실습

지도 작성 실습

방향키로 로봇을 움직여 지도를 작성

- (주의) 너무 빠른 속도로 물체에 부딪히면 전복됨.
  전복된 경우 다시 매핑해야 함
- 흰색 픽셀: Free Space로 센서에 물체가 감지되지 않는 빈 공간
- 검은색 픽셀: Occupied Space로 정적인 물체(벽, 의자, 책상) 등으로
  센서에 물체가 감지된 공간을 의미
- Occupancy Grid Mapping 방식
  : 로봇이 이동할수록 더 정확한 매핑이 가능함.
  센서의 노이즈로 인해 불확실한 정보를 확률적으로 업데이트하며
  지도를 작성함

NAV2 실습

지도 작성 실습

- 벽과 빈 공간이 모두 잘 매핑되었을 때 지도 작성을 종료

터미널 1

```
$ source ~/ros2_ws/install/local_setup.bash
$ ros2 run nav2_map_server map_saver_cli -f ~/map
```



NAV2 실습

지도 작성 실습

지도가 잘 작성되었는지 확인해 보기

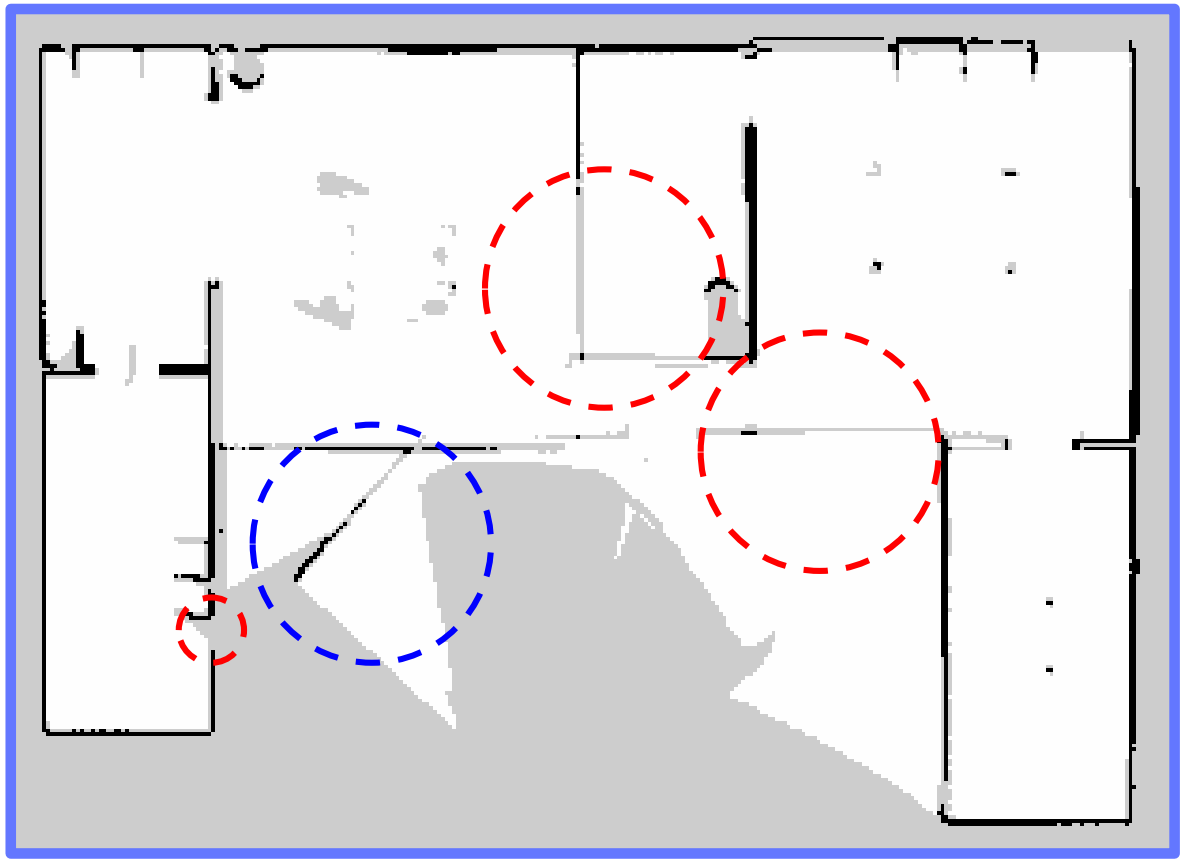
```
$ sudo apt install imagemagick
$ display ~/map.pgm
```



NAV2 실습

지도 작성 실습

- 다양한 문제가 존재
  - 벽이 정상적으로 매핑되지 않음(빨간색)
  - 로봇이 넘어져서 벽으로 매핑됨(파란색)
  - 회색 공간이 아직도 남아 있음
- 대부분 맵은 후처리를 해줘야 함



NAV2 실습

지도 작성 실습

후처리

- 벽이 있던 공간은 검은색으로 칠해주기
- 로봇이 넘어져 벽으로 인식된 건 흰색으로 칠해주기
- 회색 부분은 맵과 비교해 검은색 흰색 둘 중 하나 칠해주기

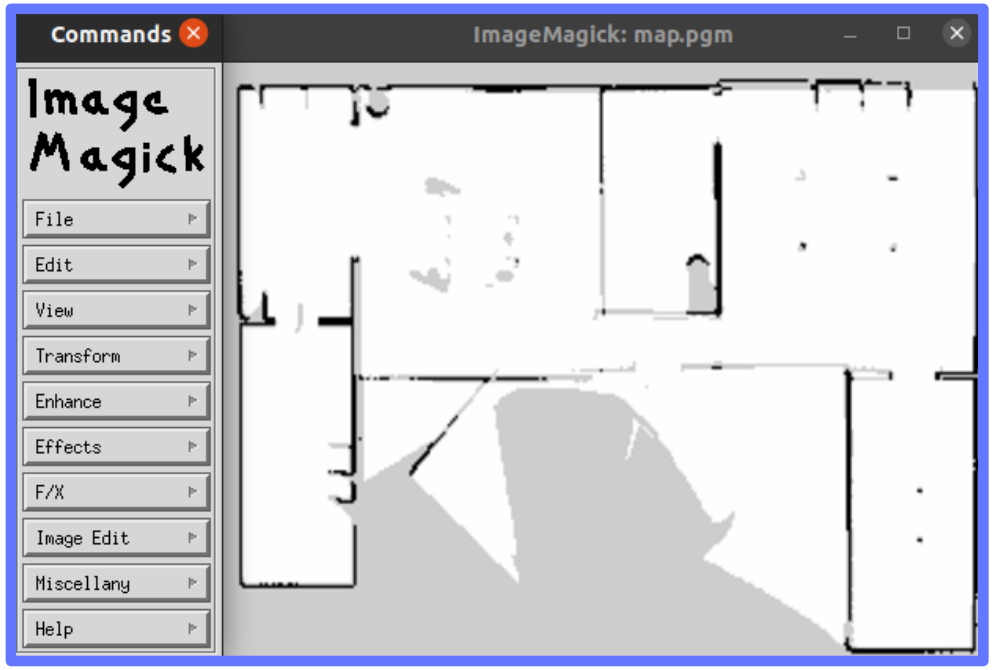
NAV2 실습

지도 작성 실습

Image Magick(그림판)으로 후처리

```
$ cd ~/
$ display ~/map.pgm
```

- 이미지에 왼쪽 마우스 클릭하면 commands 나타남
- [Image Edit] - [draw] - [color] 선택 후 후처리 진행하기
- 진행 완료 후 [dismiss] - [save] 기존 이미지에 덮어쓰기



NAV2 실습

지도 작성 실습

후처리 전과 후

